https://colab.research.google.com/drive/16x1zhUNgpShBtv8Jfdmu-7AaDsYivjQZ

1 Forecasting Future Cash Flow.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Sample historical cash flow data (monthly cash flow in thousands)
data = {
    'Month': pd.date_range(start='2023-01-01', periods=24, freq='M'),
    'CashFlow': [
        120, 130, 125, 140, 135, 150, 155, 160, 170, 165, 175, 180,
        185, 190, 195, 200, 210, 215, 220, 225, 230, 240, 245, 250
    ]
}

# Create a DataFrame
df = pd.DataFrame(data)
df.set_index('Month', inplace=True)

# Plot historical data
df.plot(title="Historical Monthly Cash Flow", ylabel="Cash Flow (in thousands)")
plt.show()

# Fit the Exponential Smoothing model
model = ExponentialSmoothing(df['CashFlow'], trend='add', seasonal=None,
initialization_method='estimated')
fit = model.fit()
```

```
# Forecast the next 6 months
forecast = fit.forecast(6)
print("Forecasted Cash Flow for next 6 months:")
print(forecast)

# Plot the historical data and forecast
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['CashFlow'], label='Historical Cash Flow')
plt.plot(forecast.index, forecast, label='Forecasted Cash Flow', marker='o')
plt.title('Cash Flow Forecast')
plt.xlabel('Month')
plt.ylabel('Cash Flow (in thousands)')
plt.legend()
plt.show()
```

2 Determining Staffing Needs Entertainment & Hospitality sector.

```
import math

def calculate_staffing_needs(expected_guests, avg_service_time_min, operating_hours,
guests_per_staff_per_hour):

    total_service_time_hours = (expected_guests * avg_service_time_min) / 60
    staff_needed = total_service_time_hours / operating_hours / (guests_per_staff_per_hour / 1)
    return math.ceil(staff_needed)

def main():
    print("Staffing Needs Calculator for Entertainment & Hospitality Sector")

    expected_guests = int(input("Enter expected number of guests: "))
    avg_service_time_min = float(input("Enter average service time per guest (in minutes): "))
```

```
    operating_hours = float(input("Enter operating hours: "))
    guests_per_staff_per_hour = int(input("Enter how many guests one staff member can handle per hour: "))

    staff_required = calculate_staffing_needs(expected_guests, avg_service_time_min, operating_hours, guests_per_staff_per_hour)

    print(f"\nEstimated number of staff required: {staff_required}")

if __name__ == "__main__":
    main()
```

```
Staffing Needs Calculator for Entertainment & Hospitality Sector
Enter expected number of guests: 10
Enter average service time per guest (in minutes): 2
Enter operating hours: 1
Enter how many guests one staff member can handle per hour: 20

Estimated number of staff required: 1
```

3 Predict Behavioral Target of a consumer.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Sample dataset (You can replace with your own CSV)
data = {
    'age': [22, 35, 26, 45, 52, 23, 40, 36, 28, 30],
    'income': [25000, 60000, 35000, 80000, 120000, 28000, 70000, 62000, 40000, 45000],
```

```
    'visits_last_month': [5, 1, 3, 2, 0, 7, 1, 2, 3, 4],
    'time_on_site': [30, 5, 15, 7, 2, 40, 8, 10, 12, 20],
    'clicked_ad': [1, 0, 1, 0, 0, 1, 0, 0, 1, 1],
    'purchased': [1, 0, 1, 0, 0, 1, 0, 0, 1, 1]
}

df = pd.DataFrame(data)

# Step 2: Feature selection
X = df[['age', 'income', 'visits_last_month', 'time_on_site', 'clicked_ad']]
y = df['purchased']

# Step 3: Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Model training
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Prediction
y_pred = model.predict(X_test)

# Step 6: Evaluation
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Optional: Feature importance
importances = model.feature_importances_
```

```
feature_names = X.columns
sns.barplot(x=importances, y=feature_names)
plt.title("Feature Importance")
plt.show()
```

4 Predict to Prevent Malfunction of a machine.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Sample machine sensor dataset
# 1 = malfunction, 0 = normal
data = {
    'temperature': [70, 85, 90, 95, 100, 65, 75, 110, 80, 105],
    'vibration': [0.2, 0.5, 0.6, 0.7, 1.0, 0.1, 0.3, 1.2, 0.4, 1.1],
    'sound_level': [30, 40, 45, 50, 60, 25, 35, 70, 38, 65],
    'rpm': [1000, 1100, 1200, 1300, 1400, 950, 1050, 1600, 1150, 1500],
    'malfunction': [0, 0, 0, 1, 1, 0, 0, 1, 0, 1]
}

df = pd.DataFrame(data)

# Step 2: Feature and target split
X = df[['temperature', 'vibration', 'sound_level', 'rpm']]
y = df['malfunction']
```

```
# Step 3: Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Train RandomForest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Predict on test set
y_pred = model.predict(X_test)

# Step 6: Evaluate performance
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Feature Importance Plot
importances = model.feature_importances_
feature_names = X.columns
sns.barplot(x=importances, y=feature_names)
plt.title("Sensor Feature Importance")
plt.show()
```

5 Predict for Early Detection of Allergic Reactions.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
import matplotlib.pyplot as plt

# Step 1: Simulated dataset
data = {
    'heart_rate': [72, 85, 95, 110, 65, 98, 100, 105, 76, 88],
    'respiratory_rate': [16, 18, 22, 30, 14, 25, 28, 32, 17, 20],
    'skin_rash': [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    'known_allergen_exposure': [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    'swelling': [0, 0, 1, 1, 0, 1, 1, 1, 0, 1],
    'reaction_history': [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    'allergic_reaction': [0, 1, 1, 1, 0, 1, 1, 1, 0, 1]  # Target variable
}

df = pd.DataFrame(data)

# Step 2: Features and target
X = df.drop('allergic_reaction', axis=1)
y = df['allergic_reaction']

# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Model training
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Predictions
y_pred = model.predict(X_test)

# Step 6: Evaluation
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Feature importance
importances = model.feature_importances_
sns.barplot(x=importances, y=X.columns)
plt.title("Symptom Importance for Allergic Reaction Prediction")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```

6 Predict for early detection of fake.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
import nltk
import string

# Download NLTK stopwords if not already downloaded
nltk.download('stopwords')
from nltk.corpus import stopwords

# Step 1: Sample Dataset (Replace with your own or use a CSV file)
data = {
    'text': [
```

```
        "Breaking news: Scientists discover cure for cancer!",
        "Click here to win a free iPhone now!",
        "Government confirms new economic policy changes.",
        "Lose weight in 7 days with this magical pill!",
        "NASA confirms life on Mars through rover data.",
        "This is not real: Man grows wings after drinking soda.",
        "Elections results: President reelected with 60% votes.",
        "Earn $500 a day working from your bedroom!"
    ],
    'label': [0, 1, 0, 1, 0, 1, 0, 1]  # 0 = Real, 1 = Fake
}

df = pd.DataFrame(data)

# Step 2: Text Cleaning
stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = ''.join([c for c in text if c not in string.punctuation])
    words = text.split()
    return ' '.join([word for word in words if word not in stop_words])

df['clean_text'] = df['text'].apply(clean_text)

# Step 3: Feature extraction with TF-IDF
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['clean_text'])
y = df['label']

# Step 4: Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 5: Train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 6: Predictions and evaluation
y_pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Make a prediction
def predict_fake(text):
    cleaned = clean_text(text)
    vect = vectorizer.transform([cleaned])
    pred = model.predict(vect)[0]
    return "Fake" if pred == 1 else "Real"

# Example usage
new_text = "Win a brand new car just by clicking this link!"
print(f"\nPrediction: '{new_text}' -> {predict_fake(new_text)}")
Confusion Matrix:
[[1 0]
 [2 0]]

Classification Report:
              precision    recall  f1-score   support
```

```
           0       0.33      1.00      0.50         1
           1       0.00      0.00      0.00         2

    accuracy                           0.33         3
   macro avg       0.17      0.50      0.25         3
weighted avg       0.11      0.33      0.17         3
```

Prediction: 'Win a brand new car just by clicking this link!' -> Real

7 Predict sickness of a person using behavioral symptoms.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Simulated dataset of behavioral symptoms and sickness
data = {
    'fever': [1, 0, 1, 1, 0, 0, 1, 0, 1, 0],         # 1 = yes, 0 = no
    'cough': [1, 0, 1, 0, 0, 1, 1, 0, 1, 0],
    'fatigue': [1, 0, 1, 1, 0, 0, 1, 0, 1, 0],
    'headache': [1, 0, 0, 1, 0, 0, 1, 0, 1, 0],
    'loss_of_appetite': [1, 0, 1, 1, 0, 0, 1, 0, 1, 0],
    'sick': [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]          # Target variable: 1 = sick, 0 = healthy
}
```

```
df = pd.DataFrame(data)

# Step 2: Feature and target separation
X = df.drop('sick', axis=1)
y = df['sick']

# Step 3: Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 4: Train Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = model.predict(X_test)

# Step 6: Evaluate the model
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 7: Feature importance visualization
importances = model.feature_importances_
sns.barplot(x=importances, y=X.columns)
plt.title('Feature Importance in Predicting Sickness')
plt.xlabel('Importance Score')
plt.ylabel('Behavioral Symptom')
plt.tight_layout()
plt.show()
```

8 Predict weather forecasting.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Step 1: Simulated historical daily temperatures (°C)
# You can replace this with real historical weather data (CSV)
data = {
    'day': range(1, 31),  # Days 1 to 30
    'temperature': [
        22, 21, 23, 24, 25, 23, 22, 21, 22, 24,
        26, 25, 27, 28, 29, 27, 26, 25, 24, 23,
        22, 23, 24, 25, 26, 27, 26, 25, 24, 23
    ]
}

df = pd.DataFrame(data)

# Step 2: Create features - previous day temperature to predict next day
df['temp_prev_day'] = df['temperature'].shift(1)
df.dropna(inplace=True)  # Drop first row with NaN

X = df[['temp_prev_day']]  # Feature: temp of previous day
y = df['temperature']      # Target: temp today

# Step 3: Train-test split (last 5 days for testing)
```

```
train_size = len(df) - 5
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

# Step 4: Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 5: Predict on test set
y_pred = model.predict(X_test)

# Step 6: Evaluate performance
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse:.2f}")

# Step 7: Plot results
plt.plot(df['day'][train_size:], y_test, label='Actual Temp')
plt.plot(df['day'][train_size:], y_pred, label='Predicted Temp', linestyle='--')
plt.xlabel('Day')
plt.ylabel('Temperature (°C)')
plt.title('Weather Forecasting - Temperature Prediction')
plt.legend()
plt.show()
```

9 Predict performance of Indian cricket team in any tournament.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# Step 1: Simulated historical ICC match data (India)
data = {
    'Opponent': ['Australia', 'England', 'Pakistan', 'South Africa', 'New Zealand', 'Australia', 'England', 'Pakistan'],
    'Venue': ['Home', 'Away', 'Neutral', 'Home', 'Away', 'Neutral', 'Home', 'Away'],
    'Toss_Won': ['Yes', 'No', 'Yes', 'No', 'Yes', 'No', 'Yes', 'No'],
    'Toss_Decision': ['Bat', 'Bowl', 'Bat', 'Bat', 'Bowl', 'Bowl', 'Bat', 'Bowl'],
    'Match_Format': ['ODI', 'ODI', 'T20', 'T20', 'ODI', 'T20', 'ODI', 'T20'],
    'Result': ['Win', 'Lose', 'Win', 'Lose', 'Win', 'Lose', 'Win', 'Lose']
}

df = pd.DataFrame(data)

# Step 2: Encode categorical variables
label_encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Step 3: Prepare features and target
X = df.drop('Result', axis=1)
y = df['Result']  # Encoded as 1=Win, 0=Lose

# Step 4: Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Step 5: Train a Random Forest classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
# Step 6: Evaluate the model
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=label_encoders['Result'].classes_))

# Step 7: Predict performance for a new match
new_match = {
    'Opponent': 'Australia',
    'Venue': 'Neutral',
    'Toss_Won': 'Yes',
    'Toss_Decision': 'Bat',
    'Match_Format': 'ODI'
}

# Encode new match
new_match_encoded = {col: label_encoders[col].transform([val])[0] for col, val in
new_match.items()}

# Predict
new_match_df = pd.DataFrame([new_match_encoded])
pred_encoded = model.predict(new_match_df)[0]
pred_label = label_encoders['Result'].inverse_transform([pred_encoded])[0]

print(f"\nPredicted result for new match: {pred_label}")
```

```
Accuracy: 0.50
Classification Report:
              precision    recall  f1-score   support

        Lose       1.00      0.50      0.67         2
```

```
        Win       0.00      0.00      0.00         0

   accuracy                           0.50         2
  macro avg       0.50      0.25      0.33         2
weighted avg       1.00      0.50      0.67         2
```

```
Predicted result for new match: Win
```

10 Predict employee growth in HR.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

# Sample data
data = {
    'Years_at_Company': [1, 3, 5, 2, 6, 4],
    'Performance_Rating': [3, 4, 5, 2, 5, 4],
    'Previous_Promotions': [0, 1, 2, 0, 2, 1],
    'Growth': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']  # Target variable
}

df = pd.DataFrame(data)

# Encode target variable
le = LabelEncoder()
df['Growth'] = le.fit_transform(df['Growth'])  # No=0, Yes=1
```

```
# Features and target
X = df[['Years_at_Company', 'Performance_Rating', 'Previous_Promotions']]
y = df['Growth']

# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")

# Predict growth for a new employee
new_employee = [[4, 4, 1]]  # Years_at_Company=4, Performance_Rating=4, Previous_Promotions=1
growth_pred = model.predict(new_employee)[0]
growth_label = le.inverse_transform([growth_pred])[0]

print(f"Predicted growth for new employee: {growth_label}")
Accuracy: 1.00
Predicted growth for new employee: Yes
```

11 Detecting intrusion in any network

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Sample network traffic data (packet_size in bytes, connection_duration in seconds)
data = {
    'packet_size': [500, 600, 1500, 2000, 100, 400, 1800, 1200, 300, 2500],
    'connection_duration': [30, 45, 10, 5, 100, 40, 8, 15, 70, 3],
    'intrusion': [0, 0, 1, 1, 0, 0, 1, 1, 0, 1]  # 0 = normal, 1 = intrusion
}

# Load data into a DataFrame
df = pd.DataFrame(data)

# Features and target
X = df[['packet_size', 'connection_duration']]
y = df['intrusion']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize the Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Example: Predict intrusion on new network traffic data
new_traffic_samples = [
    [1600, 7],   # likely intrusion
    [350, 50],   # likely normal
    [2200, 3],   # likely intrusion
]

predictions = model.predict(new_traffic_samples)

for sample, pred in zip(new_traffic_samples, predictions):
    status = "Intrusion" if pred == 1 else "Normal"
    print(f"Traffic with packet_size={sample[0]} and duration={sample[1]} is predicted as: {status}")
```

```
Accuracy: 1.0

Confusion Matrix:
 [[3]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         3

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3

Traffic with packet_size=1600 and duration=7 is predicted as: Intrusion
Traffic with packet_size=350 and duration=50 is predicted as: Normal
Traffic with packet_size=2200 and duration=3 is predicted as: Intrusion
```

12 Predicting buying behavior in retail.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score

# Sample data: age, income, past purchase (1=yes,0=no), and buying behavior (1=will buy, 0=won't buy)
data = {
    'age': [22, 35, 58, 45, 30, 40, 23, 50, 36, 28],
    'income': [30000, 60000, 80000, 70000, 40000, 50000, 32000, 75000, 62000, 41000],
    'past_purchase': [1, 0, 1, 0, 1, 1, 0, 1, 0, 0],
    'will_buy': [0, 1, 1, 0, 1, 1, 0, 1, 0, 0]
}

# Load data into DataFrame
df = pd.DataFrame(data)

# Features and target variable
X = df[['age', 'income', 'past_purchase']]
y = df['will_buy']

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize Logistic Regression model
model = LogisticRegression()
```

```
# Train the model
model.fit(X_train, y_train)

# Predict on test data
y_pred = model.predict(X_test)

# Evaluate the model
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Predict buying behavior for a new customer
new_customer = [[40, 50000, 0]]  # age=40, income=50k, no past purchase
prediction = model.predict(new_customer)

print(f"Prediction for new customer: {'Will Buy' if prediction[0] == 1 else 'Will Not Buy'}")
```

```
Accuracy: 0.67

Classification Report:
              precision    recall  f1-score   support

           0       0.50      1.00      0.67         1
           1       1.00      0.50      0.67         2

    accuracy                           0.67         3
   macro avg       0.75      0.75      0.67         3
weighted avg       0.83      0.67      0.67         3

Prediction for new customer: Will Buy
/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465:
ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```